

Competitive Programming

Section 21: Graphs I Mixed Contest

Competitive Programming Course

2026-07-05

Table of contents

1	Why a mixed graph contest now	3
2	The graph pattern map	4
3	Contest triage routine	5
4	Primer: DSU plus Kruskal	6
5	Primer: topological DP	7
6	Primer: 0-1 BFS	8
7	Modeling reversed one-way roads	9
8	Local contest problems	10
8.1	A. Existing Network	10
8.2	B. Build Timeline	10
8.3	C. One-Way Reversals	11
9	Choosing between the shortest path tools	12
10	C++ implementation notes	13
11	Python implementation notes	14
12	Proof habits for mixed graph problems	15
13	Common wrong approaches	16
14	Local judge workflow	17
15	Submission checklist	18
16	Practice queue	19

1 Why a mixed graph contest now

Sections 15 through 20 built the first graph toolbox:

- BFS for unweighted shortest paths;
- DFS for components, cycles, and structural graph exploration;
- topological sorting and dynamic programming on DAGs;
- DSU and Kruskal's algorithm;
- Dijkstra for nonnegative weighted shortest paths;
- 0-1 BFS for shortest paths with only weights 0 and 1.

In a real contest, the statement does not say “use Kruskal” or “this is 0-1 BFS”. The skill is to read the constraints, recognize the graph model, choose the cheapest correct tool, and then prove that the model really matches the statement.

This section is a mixed contest. The main concept is not a new data structure. The concept is graph diagnosis.

```
statement -> graph model -> known invariant -> implementation
```

2 The graph pattern map

Use this table as a triage tool. It is not a substitute for a proof.

Statement signal	Likely tool
minimum number of edges from a source	BFS
connected components, reachability regions	DFS or DSU
“must happen before”, dependencies, prerequisites	topological sorting
earliest finish time after prerequisites	topological DP
cheapest way to connect all vertices	MST / Kruskal
existing free connections plus paid edges	DSU first, then Kruskal
nonnegative weighted shortest path	Dijkstra
all edge costs are 0 or 1	0-1 BFS
reverse a directed edge at cost 1	0-1 BFS on a doubled edge model

When two signals appear together, do not panic. Decompose:

```
existing network -> components already connected -> DSU
pay to add cables -> choose cheapest connecting edges -> Kruskal
```

3 Contest triage routine

For each graph problem, answer these questions before coding:

1. What are the states?
2. What are the edges?
3. Are edges directed or undirected?
4. Are there weights? If yes, what values can they take?
5. Is the goal reachability, ordering, connection, or shortest path?
6. What complexity do the constraints demand?

Then select the tool:

unweighted shortest path	-> BFS
0/1 weighted shortest path	-> 0-1 BFS
general nonnegative shortest	-> Dijkstra
dependencies/order	-> topological sort
connect everything cheaply	-> MST / Kruskal
dynamic connectivity snapshot	-> DSU

4 Primer: DSU plus Kruskal

A DSU stores a partition of vertices into connected components.

Operations:

```
find(x)      -> representative of x's component
union(a,b)   -> merge components if different; return whether merge happened
```

Kruskal's algorithm sorts candidate undirected edges by weight and takes an edge exactly when it connects two different components.

Invariant:

```
After processing all edges of weight <= w, the chosen edges connect components
as cheaply as possible using only those processed edges.
```

For a graph that already has free edges, merge all free edges first. Then Kruskal starts from the real initial components.

ASCII picture:

free edges:

```
1---2      3      4---5
```

DSU components:

```
{1,2}      {3}      {4,5}
```

paid edges considered by increasing cost:

```
2--3 cost 4   connects {1,2} with {3}      take it
3--4 cost 7   connects {1,2,3} with {4,5}   take it
2--5 cost 10  now inside one component      skip it
```

5 Primer: topological DP

A directed acyclic graph, or DAG, is a directed graph with no directed cycle. Topological order lists every vertex after all its prerequisites.

Kahn's algorithm builds such an order using indegrees:

```
indeg[v] = number of incoming edges not yet removed
queue = all vertices with indeg 0

while queue not empty:
    pop u
    for each u -> v:
        remove that edge by decrementing indeg[v]
        if indeg[v] becomes 0, push v
```

If fewer than n vertices are popped, the graph has a cycle.

For earliest finish times:

```
start[v] = earliest day v may start
finish[v] = start[v] + duration[v]

for every dependency u -> v:
    start[v] = max(start[v], finish[u])
```

The `max` is essential. A task can start only after all prerequisites finish, so the slowest prerequisite controls the start day.

Small trace:

```
duration:
1:3  2:2  3:4  4:6  5:1

dependencies:
1 -> 3
2 -> 3
3 -> 4
2 -> 5
5 -> 4

finish[1] = 3
finish[2] = 2
start[3] = max(3, 2) = 3, finish[3] = 7
start[5] = 2, finish[5] = 3
start[4] = max(finish[3], finish[5]) = 7, finish[4] = 13
```

6 Primer: 0-1 BFS

0-1 BFS is shortest path for edge weights exactly 0 or 1.

Use a deque:

```
cost 0 relaxation -> push to front
cost 1 relaxation -> push to back
```

Template:

```
dist[start] = 0
deque = [start]

while deque not empty:
    u = pop front
    for edge u -> v with cost w in {0,1}:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            if w == 0: push_front(v)
            else:      push_back(v)
```

The deque behaves like a priority queue specialized to two edge costs. Cost-0 moves remain in the current distance layer; cost-1 moves go to the next layer.

7 Modeling reversed one-way roads

Suppose the input has a directed road:

```
u -> v
```

You may:

- travel from u to v for cost 0;
- travel from v to u by paying one reversal, cost 1.

So one input road becomes two graph edges:

```
u -> v cost 0  
v -> u cost 1
```

Then the answer is the shortest 0/1 path from city 1 to city n.

Example:

```
roads:  
2 -> 1  
3 -> 2  
  
to go 1 -> 3:  
1 -> 2 costs 1, because it reverses 2 -> 1  
2 -> 3 costs 1, because it reverses 3 -> 2  
answer = 2
```

8 Local contest problems

8.1 A. Existing Network

Signal:

```
already connected for free  
pay for additional weighted undirected edges  
make all cities connected
```

Tool:

```
DSU for the free edges  
Kruskal for paid edges
```

This is not Dijkstra. We are not routing from one city to another; we are buying a set of edges so the whole graph becomes connected.

8.2 B. Build Timeline

Signal:

```
dependencies  
earliest finish time  
cycle means impossible
```

Tool:

```
Kahn topological sorting  
DP over the topological order
```

This is not a shortest path problem. The edge $a \rightarrow b$ means “a must finish before b starts”, not “travel from a to b”.

8.3 C. One-Way Reversals

Signal:

```
move along direction for free  
move against direction costs one  
minimize number of reversals
```

Tool:

```
0-1 BFS
```

This is not ordinary BFS because a cost-0 move and a cost-1 move are not equal. It is not full Dijkstra because the only costs are 0 and 1.

9 Choosing between the shortest path tools

Edge costs	Tool
every edge has cost 1	BFS
every edge has cost 0 or 1	0-1 BFS
nonnegative integers such as 3, 10, 1000	Dijkstra
negative edges	future material; do not use Dijkstra

The phrase “minimum” does not automatically mean Dijkstra. First inspect the cost structure.

10 C++ implementation notes

Use these standard containers:

```
vector<vector<int>>> adj;           // unweighted graph
vector<vector<pair<int,int>>>> adj; // weighted graph
queue<int> q;                      // Kahn / BFS
deque<int> dq;                    // 0-1 BFS
vector<tuple<long long,int,int>>> edges; // Kruskal
```

For large answers, use `long long`. MST costs and timeline finish days can exceed `int`.

For recursion-heavy graph code, prefer iterative approaches when possible in this course phase. These local problems can all be solved iteratively.

11 Python implementation notes

Use:

```
from collections import deque
```

For fast input:

```
data = list(map(int, sys.stdin.buffer.read().split()))
```

This style is especially useful when n and m are up to $2 * 10^5$.

Avoid recursive DFS for large graphs unless you deliberately manage recursion limits and stack behavior. The local Section 21 solutions do not require it.

12 Proof habits for mixed graph problems

A mixed contest proof usually has two layers.

First, prove the model:

```
Every valid real-world solution corresponds to a graph solution of same cost.  
Every graph solution corresponds to a valid real-world solution of same cost.
```

Second, prove the algorithm on that model:

```
Kruskal chooses a minimum-cost connector.  
Kahn processes every prerequisite before its dependents.  
0-1 BFS computes shortest distances in a graph with edge costs 0 and 1.
```

If either layer is missing, bugs hide easily.

13 Common wrong approaches

Running Dijkstra for Existing Network.

There is no single source shortest path target. The problem asks for a minimum total set of connecting edges.

Ignoring the free edges in Existing Network.

Free edges must be unioned before any paid edge is considered.

Using BFS for One-Way Reversals.

BFS treats cost 0 and cost 1 edges equally, which changes the objective.

Only checking reachability for Build Timeline.

You also need earliest finish values, and cycles must produce IMPOSSIBLE.

Using min instead of max for earliest starts.

A module waits for all prerequisites, so the latest prerequisite matters.

14 Local judge workflow

Run just this section:

```
CP_TARGET=solution python3 sections/21_graphs_i_mixed_contest/check.py
```

Run the entire course judge:

```
CP_TARGET=solution python3 tools/check_all.py
```

The local judge checks both C++ and Python reference solutions against fixed tests and generated random cases.

15 Submission checklist

Before submitting a graph solution, ask:

```
Did I identify states and edges explicitly?  
Did I choose the algorithm from edge weights and objective, not just keywords?  
Are edges directed or undirected exactly as modeled?  
Do I need long long for accumulated costs?  
Did I handle disconnected/impossible cases?  
Did I prove the graph model and the algorithm separately?
```

16 Practice queue

Use the external queue in `PRACTICE.md` after solving the local set. It mixes CSES, AtCoder, Kattis, and Codeforces graph problems so the algorithm is not announced in advance.